

# Collector Deployment Patterns

Series: OTEL | Notebook: 3 of 8 | Created: January 2026

## Deploying the OpenTelemetry Collector

The OTel Collector can be deployed in various patterns depending on your infrastructure. This notebook covers deployment modes, Kubernetes configurations, and best practices for production.

### Table of Contents

- 1. Deployment Modes
- 2. Agent Mode (Sidecar/DaemonSet)
- 3. Gateway Mode
- 4. Kubernetes Deployment
- 5. Docker Compose
- 6. High Availability
- 7. Resource Sizing
- 8. Security Considerations

### Prerequisites

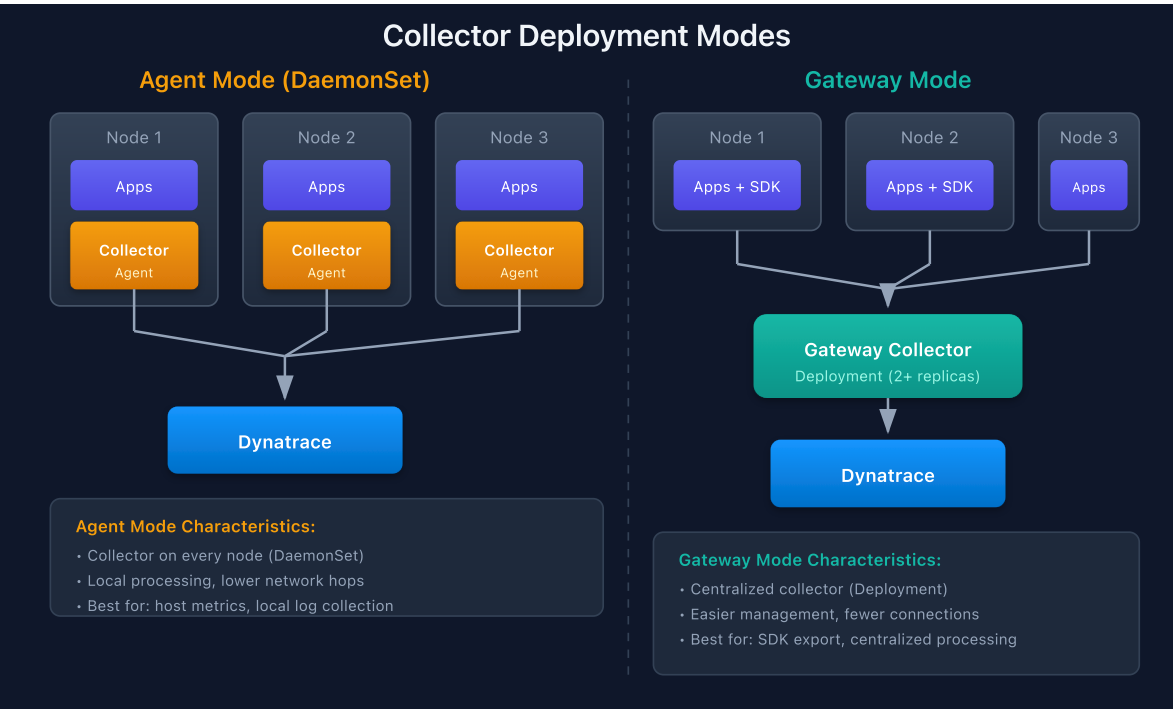
| Requirement | Details                         |
|-------------|---------------------------------|
| Knowledge   | OTEL-02: Collector Architecture |
| Tools       | kubect!, Helm, or Docker        |

# 1. Deployment Modes

## Mode Comparison

| Mode    | Deployment        | Use Case            | Pros                          | Cons                 |
|---------|-------------------|---------------------|-------------------------------|----------------------|
| Agent   | DaemonSet/Sidecar | Per-node collection | Low latency, local processing | More instances       |
| Gateway | Deployment        | Centralized         | Single point, aggregation     | Potential bottleneck |
| Hybrid  | Both              | Large scale         | Best of both                  | Complex              |

## Architecture Patterns



## 2. Agent Mode (Sidecar/DaemonSet)

---

### DaemonSet Configuration

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: otel-collector-agent
  namespace: otel
spec:
  selector:
    matchLabels:
      app: otel-collector-agent
  template:
    metadata:
      labels:
        app: otel-collector-agent
    spec:
      containers:
        - name: collector
          image: otel/opentelemetry-collector-contrib:latest
          args:
            - --config=/conf/otel-collector-config.yaml
          ports:
            - containerPort: 4317 # OTLP gRPC
            - containerPort: 4318 # OTLP HTTP
          resources:
            requests:
              cpu: 100m
              memory: 256Mi
            limits:
              cpu: 500m
              memory: 512Mi
          volumeMounts:
            - name: config
              mountPath: /conf
      volumes:
        - name: config
          configMap:
            name: otel-collector-config
```

## Sidecar Pattern

```
apiVersion: v1
kind: Pod
metadata:
  name: my-app
spec:
  containers:
    - name: app
      image: my-app:latest
      env:
        - name: OTEL_EXPORTER_OTLP_ENDPOINT
          value: http://localhost:4317
    - name: otel-collector
      image: otel/opentelemetry-collector-contrib:latest
      args:
        - --config=/conf/otel-collector-config.yaml
      ports:
        - containerPort: 4317
```

## 3. Gateway Mode

---

### Gateway Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: otel-collector-gateway
  namespace: otel
spec:
  replicas: 2 # HA
  selector:
    matchLabels:
      app: otel-collector-gateway
  template:
    metadata:
      labels:
        app: otel-collector-gateway
    spec:
      containers:
        - name: collector
          image: otel/opentelemetry-collector-contrib:latest
          args:
            - --config=/conf/otel-collector-config.yaml
          ports:
            - containerPort: 4317
            - containerPort: 4318
          resources:
            requests:
              cpu: 500m
              memory: 1Gi
            limits:
              cpu: 2000m
              memory: 4Gi
```

## Gateway Service

```
apiVersion: v1
kind: Service
metadata:
  name: otel-collector-gateway
  namespace: otel
spec:
  ports:
    - name: otlp-grpc
      port: 4317
      targetPort: 4317
    - name: otlp-http
      port: 4318
      targetPort: 4318
  selector:
    app: otel-collector-gateway
```

## 4. Kubernetes Deployment

---

### Helm Installation

```
# Add Helm repo
helm repo add open-telemetry https://open-telemetry.github.io/opentelemetry-
helm-charts
helm repo update

# Install as DaemonSet (agent mode)
helm install otel-collector open-telemetry/opentelemetry-collector \
  --namespace otel \
  --create-namespace \
  --set mode=daemonset

# Install as Deployment (gateway mode)
helm install otel-collector open-telemetry/opentelemetry-collector \
  --namespace otel \
  --create-namespace \
  --set mode=deployment \
  --set replicaCount=2
```

## Helm Values for Dynatrace

```
# values.yaml
mode: deployment
replicaCount: 2

config:
  receivers:
    otlp:
      protocols:
        grpc:
          endpoint: 0.0.0.0:4317
        http:
          endpoint: 0.0.0.0:4318

  processors:
    batch:
      timeout: 10s
    memory_limiter:
      check_interval: 1s
      limit_mib: 800

  exporters:
    otlphttp:
      endpoint: https://${DT_ENDPOINT}/api/v2/otlp
      headers:
        Authorization: Api-Token ${DT_TOKEN}

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [otlphttp]
    metrics:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [otlphttp]
    logs:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [otlphttp]
```

# 5. Docker Compose

## Basic Setup

```
# docker-compose.yaml
version: '3.8'

services:
  otel-collector:
    image: otel/opentelemetry-collector-contrib:latest
    command: ["--config=/etc/otel-collector-config.yaml"]
    volumes:
      - ./otel-collector-config.yaml:/etc/otel-collector-config.yaml
    ports:
      - "4317:4317" # OTLP gRPC
      - "4318:4318" # OTLP HTTP
      - "13133:13133" # Health check
    environment:
      - DT_ENDPOINT=${DT_ENDPOINT}
      - DT_TOKEN=${DT_TOKEN}

  my-app:
    image: my-app:latest
    environment:
      - OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector:4317
      - OTEL_SERVICE_NAME=my-app
    depends_on:
      - otel-collector
```

# 6. High Availability

## HA Considerations

| Component         | HA Strategy                            |
|-------------------|----------------------------------------|
| Agent (DaemonSet) | Node failure = pod restarts            |
| Gateway           | Multiple replicas behind load balancer |
| Persistent Queue  | Prevent data loss during restarts      |

## Gateway with Persistent Queue

```
exporters:
  otlphttp:
    endpoint: https://${DT_ENDPOINT}/api/v2/otlp
    sending_queue:
      enabled: true
      num_consumers: 10
      queue_size: 1000
      storage: file_storage
    retry_on_failure:
      enabled: true
      initial_interval: 1s
      max_interval: 30s
      max_elapsed_time: 300s

extensions:
  file_storage:
    directory: /var/lib/otel/storage
    timeout: 10s

service:
  extensions: [file_storage]
```

## Pod Disruption Budget

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: otel-collector-pdb
spec:
  minAvailable: 1
  selector:
    matchLabels:
      app: otel-collector-gateway
```

## 7. Resource Sizing

---

### Sizing Guidelines

| Throughput                | CPU  | Memory | Replicas |
|---------------------------|------|--------|----------|
| Low (<1000 spans/s)       | 100m | 256Mi  | 1        |
| Medium (<10k spans/s)     | 500m | 1Gi    | 2        |
| High (<100k spans/s)      | 2    | 4Gi    | 3+       |
| Very High (>100k spans/s) | 4+   | 8Gi+   | 5+       |

### Memory Limiter Tuning

```
processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 800      # 80% of container memory limit
    spike_limit_mib: 200 # Room for spikes
```

### Horizontal Pod Autoscaler

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: otel-collector-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: otel-collector-gateway
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

## 8. Security Considerations

---

### Token Management

Store Dynatrace API tokens in Kubernetes Secrets:

```
# Kubernetes Secret (create via kubectl or sealed-secrets)
apiVersion: v1
kind: Secret
metadata:
  name: dynatrace-otel-token
type: Opaque
stringData:
  token: <your-dynatrace-api-token>
```

```
# Reference in Deployment
env:
  - name: DT_TOKEN
    valueFrom:
      secretKeyRef:
        name: dynatrace-otel-token
        key: token
```

### TLS Configuration

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
        tls:
          cert_file: /certs/server.crt
          key_file: /certs/server.key
          client_ca_file: /certs/ca.crt # mTLS
```

## Network Policy

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: otel-collector-policy
spec:
  podSelector:
    matchLabels:
      app: otel-collector-gateway
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - namespaceSelector: {}
      ports:
        - protocol: TCP
          port: 4317
        - protocol: TCP
          port: 4318
  egress:
    - to:
        - ipBlock:
            cidr: 0.0.0.0/0
      ports:
        - protocol: TCP
          port: 443
```

## Summary

---

In this notebook, you learned:

- Deployment modes: Agent vs. Gateway vs. Hybrid
- DaemonSet and Sidecar configurations for agent mode
- Gateway deployment with services
- Kubernetes deployment via Helm
- Docker Compose setup
- High availability with persistent queues
- Resource sizing guidelines
- Security best practices

---

## Next Steps

---

| Next Notebook                         | Topic              |
|---------------------------------------|--------------------|
| <b>OTEL-04: Trace Instrumentation</b> | Instrumenting code |
| <b>OTEL-07: Dynatrace Integration</b> | Complete DT setup  |

---

## References

---

- [Collector Deployment](#)
  - [Helm Chart](#)
  - [Docker Images](#)
- 

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*